Data Dispersion and the Row-Wise LM Backend

Meeting preparation report

October 19, 2026

0 Project overview and current position

The thesis project develops and evaluates LM-IDNet, a lightweight and explainable anomaly detector for Internet-of-Things traffic. Its goal is to learn the ordinary behaviour of one device without using attack examples for training, then identify later windows whose traffic is unlikely under that learned behaviour.

Each supported packet is assigned to TCP, ordinary UDP, SSDP, or ARP and counted in a ten-minute window. SSDP is separated from ordinary UDP, so a packet contributes to only one category. One window becomes the row

$$\mathbf{x}_r = (x_{r,\mathrm{TCP}}, x_{r,\mathrm{UDP}}, x_{r,\mathrm{SSDP}}, x_{r,\mathrm{ARP}}).$$

Missing windows are excluded because their traffic is unknown, whereas observed silent windows are retained as zero-count rows because they record real periods with no captured device traffic.

The normal-training rows are used to fit a Dirichlet–multinomial parameter vector $\boldsymbol{\alpha}$. Its expected protocol share for category k is α_k/α_0, where $\alpha_0 = \sum_k \alpha_k$, and its concentration allows the protocol mixture to vary between otherwise normal windows. Once the model is fitted, the detector evaluates whether each later row is plausible under that learned behaviour.

The implemented pipeline is:

packet captures $\longrightarrow$ validated device windows $\longrightarrow$ normal-model fitting $\longrightarrow$ benign calibration

$\longrightarrow$ static or adaptive scoring $\longrightarrow$ labelled evaluation and audit reports

The anomaly threshold is calibrated on a separate benign period, and attack labels are used only after scoring to evaluate the decisions. This separation keeps model fitting unsupervised and makes every result traceable to its input data, fitted model, and threshold.

The current labelled development results show high precision but incomplete recall. Table 1 gives the static baseline and the selected periodic-refit candidate. The adaptive candidate improves Chromecast substantially, but it does not improve Samsung over the static baseline.

These are development results, not a locked final evaluation. D-Link has no attack labels in this project and is therefore used for ψ and numerical comparisons rather than detection metrics. The remaining thesis work includes stronger temporal validation, safer adaptation, and evaluation of the frozen detector on untouched data.

Table 1: Current labelled development results. The periodic-refit candidate uses a 288-window update interval and requires 240 accepted windows.

Dataset	Detector	Precision	Recall	F1
Chromecast	Static	88.24%	46.88%	61.22%
Chromecast	Periodic refit	92.00%	71.88%	80.70%
Samsung Camera	Static	92.31%	68.57%	78.69%
Samsung Camera	Periodic refit	93.88%	65.71%	77.31%

0.1 Scope of the following sections

The complete project includes capture validation, preprocessing, calibration, scoring, adaptation, and evaluation. The rest of this report deliberately focuses on the part of that pipeline related to the Languasco–Migliardi (LM) algorithm: the fitted-ψ regime in which it is used, the row-wise likelihood required by the detector, the integration changes made around the native code, and the resulting runtime and numerical evidence.

LM is a numerical backend for evaluating the Dirichlet–multinomial likelihood; it is not the detector by itself and does not define the fitted ψ or the paper's threshold. The project results above provide the wider thesis context, while the following sections explain why the backend receives separate windows and how its behaviour was validated within the implemented detector.

I The fitted ψ values in the three datasets

The project uses ψ as the fitted Dirichlet–multinomial dispersion parameter. LM evaluates the likelihood for the supplied positive $\boldsymbol{\alpha}$ and corresponding ψ. Its numerical method is especially relevant when ψ is small: the usual subtraction of large, nearby log-gamma values is then more likely to lose precision. The $\psi < 0.05$ threshold discussed below is the empirical rule defined by the baseline paper; it is not a prerequisite for calling LM.

Before discussing how the LM code was integrated, it helps to establish the data received by the backend. All three devices use the row representation defined in Section 0, but they differ in their amount of window-to-window variation and in the ground truth available for interpreting that variation.

The D-Link camera dataset is used as the reference baseline for this comparison and is also the most stable of the three datasets considered here. It has no attack labels in this project. Its windows can be described statistically, but they cannot be separated into confirmed benign and attack groups. This point matters: "unlabeled" means that the ground truth is unavailable; it is not a claim that every D-Link window is benign. Chromecast and Samsung Camera come from the UNSW IoT attack traces. Their window-level annotations identify the windows that overlap a recorded attack, which allows a descriptive comparison between benign-labelled and attack-labelled traffic.

Table 2: Data available for the fitted-ψ comparison. Counts refer to observed ten-minute windows in the processed data.

Dataset	Days	Windows	Silent	Attack-labelled
D-Link camera	12	1,692	0	–
Chromecast	17	2,349	187	38
Samsung Camera	12	1,645	49	82

I.1 The Dirichlet–multinomial dispersion parameter

The model is Dirichlet–multinomial. Its positive parameter vector is $\boldsymbol{\alpha}$, its total concentration is

$$A = \sum_k \alpha_k,$$

and the project reports the reciprocal concentration

$$\psi = \frac{1}{A}.$$

The following comparisons use this fitted parameter directly. Smaller ψ means larger concentration and a greater chance that a conventional log-gamma likelihood calculation will lose numerical precision.

The ICNC 2025 paper (pp. 28–32, doi: 10.1109/ICNC64010.2025.10993688) by S. Al-Haj Baddar, A. Languasco, and M. Migliardi served as the main experimental baseline for this project. Its D-Link Camera 5 data, ten-minute four-protocol count vectors, Dirichlet–multinomial fitting, and LM comparison provided the starting point that this project extends to calibrated anomaly detection on additional devices.

That paper defines a dataset as overdispersed when $\psi < 0.05$. This report uses the same empirical threshold for direct comparison. It is a cutoff applied to the fitted ψ, not an LM precondition or a hard numerical failure boundary. Numerical risk increases progressively as ψ becomes smaller.

I.2 Training data and the complete available datasets

Table 3 shows two fitted scopes. The training scope contains only the captures selected to learn the normal model. For the two labeled datasets, those training captures contain no attack-labelled windows. The all-days scope is a separate descriptive fit over every processed day. It deliberately mixes partitions and, for the UNSW devices, includes the attack-labelled periods. It must not be interpreted as the model used by the detector.

The D-Link values and the Chromecast training value satisfy the paper's $\psi < 0.05$ condition. The Chromecast all-days value and both Samsung values are above it. D-Link has the smallest fitted ψ and therefore the greatest numerical risk among these aggregate fits. The threshold classifies the fitted values according to the paper; within either side of the threshold, numerical risk continues to increase as ψ decreases.

Table 3: Fitted Dirichlet–multinomial ψ by scope.

Dataset	Normal training scope	All available days
D-Link camera	0.008182	0.006833
Chromecast	0.035589	0.076318
Samsung Camera	0.057491	0.073784

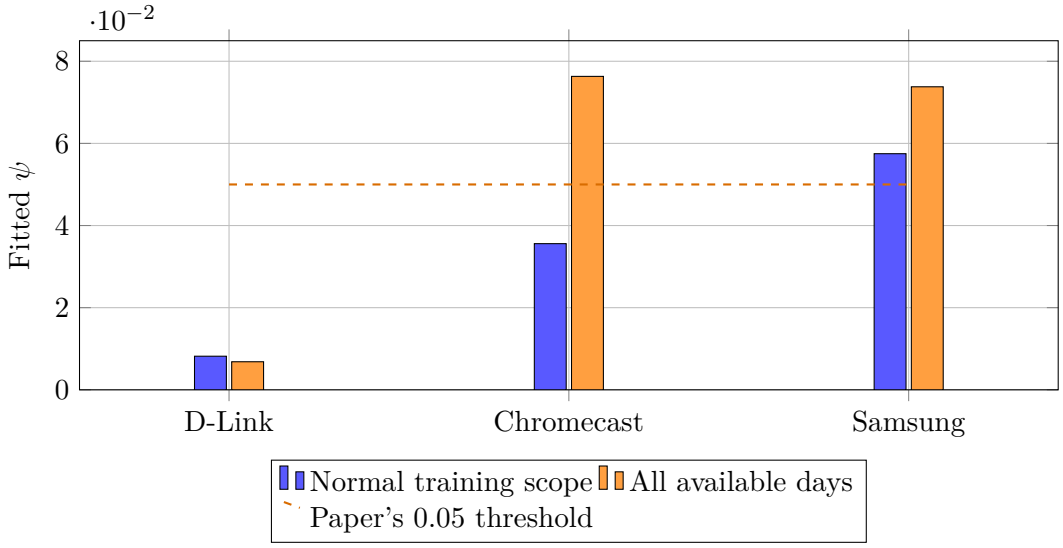


Figure 1: The D-Link estimate remains low when all available days are combined. The whole-dataset estimates rise for both labeled datasets, where the combined scope includes temporal changes and attack-labelled windows.

I.3 What the labels add to the comparison

For Chromecast and Samsung, the labels allow the complete collection of rows to be split into benign-labelled and attack-labelled subsets. Table 4 is descriptive: each value is a new fit to the stated subset. The attack fits use only 38 Chromecast windows and 82 Samsung windows, and each subset combines several attack types, so these numbers should not be treated as universal attack parameters.

Table 4: Whole-dataset fits separated by the available window labels.

Dataset	All rows	Benign-labelled	Attack-labelled	Attack windows
D-Link camera	0.006833	–	–	–
Chromecast	0.076318	0.060307	0.577558	38
Samsung Camera	0.073784	0.023943	0.656396	82

For both labeled devices, the attack-only estimate is much larger than the benign-only estimate. In these data, the attack windows therefore contain a far less stable protocol composition under a single fitted Dirichlet–multinomial model. The all-rows value lies between the two subset fits and also reflects changes between days. D-Link has no corresponding split, so its low all-days value cannot be used as evidence about attack traffic; it is simply the value fitted to the unlabeled collection available to the project.

I.4 What the individual days reveal

The aggregate fitted ψ values hide substantial day-to-day changes. Figure 2 plots the fitted ψ for each day on a logarithmic vertical scale. The horizontal position is the chronological capture number within each device, not a shared calendar date. Red triangles mark days containing at least one attack-labelled window. The exact dates and values appear in Table 5.

The D-Link days form a comparatively narrow band: eleven of the twelve lie between about 0.0032 and 0.0080, with October 8 at 0.0147. Chromecast varies more, including two completely silent days for which ψ cannot be estimated. Its largest daily value, 0.1967, occurs on October 23, which contains 32 attack-labelled windows. Samsung changes most sharply: its daily values range from about 0.00088 to 0.209, and its two largest values occur on June 1 and June 2, which contain 24 and 42 attack-labelled windows.

The 0.05 line applies the paper's condition to each daily fit. Values far below that line, especially the smallest Samsung values, are also the cases in which conventional log-gamma evaluation has the greatest chance of losing precision. The line is a paper-defined cutoff, while the numerical risk changes continuously with ψ.

The labels make those coincidences interpretable, but they do not prove that every high daily value is caused by an attack. Chromecast October 16 reaches 0.0974 without a labeled attack, Samsung May 28 reaches 0.1472 without one, and some attack-bearing days have much smaller values. A daily fit summarizes all windows from that day, including benign traffic, attack traffic, changes in activity, and differences in packet volume. It is therefore evidence about the day's heterogeneity, not a detector output.

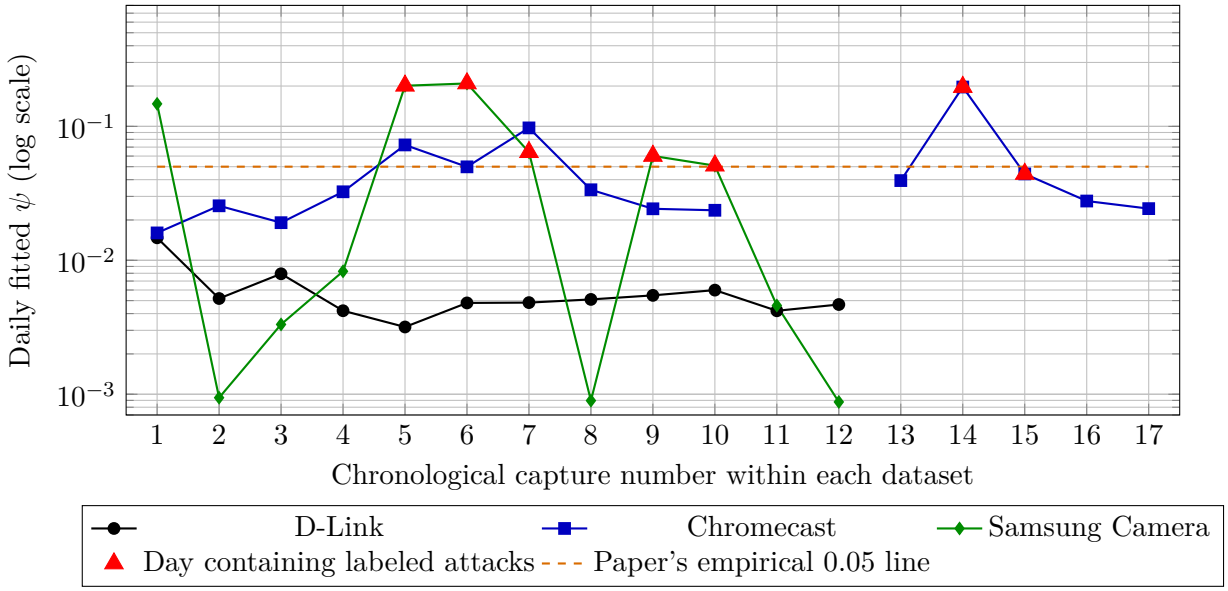


Figure 2: Daily fitted Dirichlet–multinomial dispersion-parameter values. The logarithmic scale is needed because the estimates span more than two orders of magnitude. A red marker describes the complete day containing attacks, not an attack-only fit.

Table 5: Daily fitted ψ. "Att." is the number of labeled attack windows on that day; D-Link has no attack labels. A dash means that every window was silent, so no protocol composition was available to fit.

D-Link camera		Chromecast			Samsung Camera		
Date	ψ	Date	ψ	Att.	Date	ψ	Att.
2020-10-08	0.014719	2018-10-10	0.016012	0	2018-05-28	0.147219	0
2020-10-09	0.005184	2018-10-11	0.025523	0	2018-05-29	0.000942	0
2020-10-10	0.007931	2018-10-12	0.019097	0	2018-05-30	0.003321	0
2020-10-11	0.004202	2018-10-13	0.032473	0	2018-05-31	0.008272	0
2020-10-12	0.003178	2018-10-14	0.072661	0	2018-06-01	0.201022	24
2020-10-13	0.004805	2018-10-15	0.049859	0	2018-06-02	0.209132	42
2020-10-14	0.004832	2018-10-16	0.097399	0	2018-06-03	0.064162	4
2020-10-15	0.005104	2018-10-17	0.033622	0	2018-06-04	0.000896	0
2020-10-16	0.005472	2018-10-18	0.024242	0	2018-06-05	0.060155	6
2020-10-19	0.005989	2018-10-19	0.023627	0	2018-06-06	0.050909	6
2020-10-21	0.004198	2018-10-20	–	0	2018-06-07	0.004597	0
2020-10-22	0.004676	2018-10-21	–	0	2018-06-08	0.000876	0
		2018-10-22	0.039374	0			
		2018-10-23	0.196667	32			
		2018-10-25	0.044145	6			
		2018-10-26	0.027706	0			
		2018-10-27	0.024312	0			

The main conclusion for the rest of this report is that rows carry real information. The daily and label-separated fits change substantially when different collections of windows are used. Pooling every row into one count vector would retain the grand protocol totals but erase precisely the window-to-window and day-to-day variation that the Dirichlet–multinomial model is intended to represent in this project.

II Why the project evaluates windows separately with the LM backend

II.1 What the rows mean, and why keeping them matters

The project treats each window as a separate observation from the same normal device model. In practical terms, the model is meant to learn from a sequence of window-sized snapshots, not just from one large bag of packets. Combining the rows first would preserve the overall totals for TCP, UDP, SSDP, and ARP, but it would erase which mixture occurred in which window. That distinction matters when ordinary windows vary and when later decisions are made for individual windows. This is the project's likelihood assumption; it does not by itself prove that neighbouring windows are statistically independent. It says that, for this fitting objective, each window contributes its own observation to the shared device model.

Here is a small two-category example. The real project has four categories; two are used here only to make the arithmetic easy to follow. Suppose two windows contain

$$\mathbf{x}_1 = (2, 0), \qquad \mathbf{x}_2 = (1, 1).$$

The first window contains only category A; the second contains both A and B. If the rows are pooled, the only vector left is

$$\mathbf{x}_1 + \mathbf{x}_2 = (3, 1).$$

The pooled vector says that the dataset had three A counts and one B count. It does not say that B was absent from the first window and present in the second. For a model of window-level behaviour, that lost pattern is useful evidence about how much windows vary.

The project therefore evaluates each row under the current model and adds the log-likelihood contributions:

$$\ell_{\text{rows}}(\boldsymbol{\alpha}) = \ell(\mathbf{x}_1; \boldsymbol{\alpha}) + \ell(\mathbf{x}_2; \boldsymbol{\alpha}).$$

This is the usual way to combine log-likelihoods when treating observations as separate samples: their probabilities multiply, and taking the logarithm turns that product into a sum. The rows themselves are not added together.

The difference can be seen even with $\boldsymbol{\alpha} = (1, 1)$. The parameter-dependent part of the Dirichlet–multinomial log-likelihood used by the backends is

$$K(\mathbf{x}; \boldsymbol{\alpha}) = \log\Gamma(\alpha_0) - \log\Gamma(\alpha_0 + N) + \sum_k \left[\log\Gamma(\alpha_k + x_k) - \log\Gamma(\alpha_k)\right], \qquad N = \sum_k x_k.$$

The backend leaves out the multinomial coefficient because it depends only on the observed counts, not on alpha. For these rows, the calculations are:

Calculation	What is evaluated	Result
Evaluate rows, then add	$K((2,0);\alpha) + K((1,1);\alpha)$	-2.89037
Add counts, then evaluate	$K((3,1);\alpha)$	-2.99573

These are different because they answer different questions: the first evaluates two observed windows, while the second evaluates one pooled count vector. The example is not a claim that one number is inherently more accurate; it illustrates that pooling changes the likelihood being calculated. Leaving out the count-dependent coefficient does not affect which alpha fits these same rows best; the scoring code adds that coefficient when it needs the full single-window log-probability.

If the only scientific question were the device's total protocol composition over a long period, pooling could be a reasonable summary. This detector also models variation between windows and scores individual windows, so the project keeps the windows separate for its likelihood objective. It still combines their evidence into one dataset score by summing their likelihood values. That final scalar is sufficient for fitting one shared alpha; the important part is that each row affects the score through its own likelihood calculation before the contributions are combined.

II.2 How the professor's capsule handles the rows

The original capsule does not discard the rows from all of its fitting logic. Its Minka fixed-point update reads the original matrix `D_counts` and loops over its rows when it forms the update. For a given category, it adds the row-specific digamma contributions; it also uses each row's total count in the concentration update. Thus the update step is based on the individual rows.

The difference is in the likelihood calculation used by the original LM-wrapped fitting path. Before the LM likelihood is called, the helper `find_X(D_counts)` adds each category's counts down the rows. For the example above, it turns the two rows into the single vector $(3, 1)$. The fitting function then derives the category probabilities and dispersion parameter from its current alpha vector and calls the scalar C function `loggamma_LM` once for the old alpha and once for the candidate alpha. Those calls evaluate the likelihood of the pooled count vector. Their difference is used in the capsule's stopping check.

The main functions to recognize in the original source are:

`Function`	Role in the original fitting path
`load_dataset_from_file`	Reads the count matrix and prepares it for fitting.
`find_X`	Sums each category column across all rows, producing one pooled count vector.
`minka_procedure_LM_c_wrapped_mode`	Updates alpha from the separate rows, then compares LM likelihoods for the pooled vector to decide when to stop.
`loggamma_LM`	Scalar native entry point used by that fitting function for one count vector and one alpha setting.
`logL_diretta_global`	Evaluates the scalar vector's Dirichlet–multinomial log-likelihood kernel using the LM calculation.

The capsule's likelihood helper `LM_logL_dataset` also takes a single count vector `X`; it forms one total count and combines the category terms for that vector. In the C-wrapped fitting route, the corresponding single-vector call is `loggamma_LM`. The word "dataset" in the helper name can be misleading here: the likelihood call receives a category-total vector, not the original matrix of separate windows.

So, if the question is whether the professor's LM likelihood call sums the individual row likelihoods, the answer for this original fitting path is no. It evaluates one likelihood for the column-summed vector. If the question is whether the professor's entire estimator uses the rows, the answer is yes: the Minka updates do. The project changed the likelihood objective used with those updates so that it also represents the rows separately.

II.3 How the project calls the LM code

The project has its own estimator. Its Minka fixed-point updates are implemented in Python in `DirichletMultinomialEstimator`; they use the rows of the training matrix. The estimator asks a common likelihood interface to evaluate the current and updated alpha values. That interface can use either SciPy or the LM backend.

With the LM backend selected, the call travels through `LmLogLikelihood.calculate`, then the Python-to-C bridge `NativeLmKernel.calculate`, and finally into the native matrix function `loggamma_LM_matrix`. The original scalar LM calculation remains the numerical engine for an individual row. The project-side matrix function accepts all rows, evaluates each one with the same alpha parameters, and returns the sum of their likelihood-kernel values.

In outline, the project's calculation is:

```
total = 0
for each window row:
    total += LM_likelihood_for_one_row(row, alpha)
return total
```

The important detail is the order: evaluate a row first, then add the resulting likelihood value. It does not add the count rows first.

II.4 Why the project-side LM integration was adapted

The professor's scalar LM entry point was designed to calculate the likelihood for one count vector at a time. The project's estimator supplies a matrix because its fitting objective treats each time window as its own observation. The first project-side adaptation added a matrix entry point that bridges those two shapes: it accepts the matrix, sends each row through the existing scalar LM calculation, and adds the returned values. It also checks that the inputs and numerical results are valid and cleans up the native workspace on success or failure. The original files in `lm-algorithm/code/` were left unchanged; the application-facing changes are in the project's copy under `src/lm_idnet/algorithms/native/`.

That first version repeated setup for every row. The LM routine needed values derived from alpha and the requested precision, and it searched for an Euler–Maclaurin expansion order and a horizontal

shift that met the requested error bound. Within one matrix call, those setup inputs are the same for every row. Repeating the setup did not make the rows more accurate; it did the same work again.

The later optimization moved the shared setup before the row loop. It prepares the native workspace and selects the expansion order and horizontal shift once, then evaluates each row separately using that shared setup, adds the row results, and releases the workspace. Checks that depend on a row's counts remain inside the loop. In outline:

```
prepare LM settings for this alpha and precision
total = 0
for each window row:
    check this row
    total += LM_likelihood_for_one_row_using_prepared_settings(row)
release LM workspace
return total
```

This optimization does not pool rows. It also does not change the LM approximation, its error-control rules, Minka's update, or the row-wise likelihood being computed. The change record reports that the old and optimized project libraries returned identical values across 1,000 tested matrices. In that controlled comparison, the 846-row likelihood call became 51.5% faster. Those are results for the project's matrix calculation and workload; they do not establish that every LM operation is faster than SciPy.

II.5 A concise explanation for the meeting

The project represents a device's traffic as one four-category count row per time window, because it needs to model normal variation between windows and score windows individually. The professor's original LM-wrapped fit also uses the individual rows in the Minka update, but it sums the count columns before calling its scalar LM likelihood for the stopping comparison. The project instead passes the rows to its LM backend, calculates each row's likelihood contribution, and sums those values, which is the dataset log-likelihood for separate observations. The project-side LM adapter was added to make the professor's one-vector numerical routine accept that matrix calculation; its subsequent optimization reuses alpha- and precision-dependent setup across the rows. It preserves the same row-wise result while avoiding repeated setup.

II.6 The other native changes, in one minute

The row-wise interface and its setup optimization are the changes that need the detailed explanation above. The remaining changes to the project copy of the professor's native code can be summarized briefly:

- **Exact zero-count handling.** A zero packet count contributes exactly zero to the paired log-gamma difference, so the native function now returns that identity directly. This matters because an otherwise valid window can have zero packets in one category, and a silent window has zero in every category.

- **Reliable coefficient-table loading.** The two tables needed by the likelihood path are now loaded from explicit paths supplied by the Python adapter. Both files are validated before the active tables are replaced, and failure is returned to the caller instead of depending on the process's current directory or terminating the application.
- **Explicit native-memory ownership.** Parameter arrays and the temporary error-search workspace are checked after allocation and released on success and failure paths. This prevents repeated likelihood calls during fitting from accumulating leaked native memory.
- **Controlled library rebuild.** The modified source was rebuilt with the project's existing compiler image and optimization flags, then checked against the source and exercised through the Python adapter. This step does not change the mathematics; it ensures that the shared library actually contains the reviewed source changes.

III What was timed and what the runtime numbers mean

Runtime claims are meaningful only when the timed operation is named. This project contains several operations that all involve a likelihood, but they answer different questions and have very different costs. In particular, the reported "complete model fit" is not the complete command-line pipeline, and the time printed by the professor's original capsule is not the complete fit wall time.

The broad application flow is

$$\boxed{\begin{array}{c}\text{packet preprocessing} \longrightarrow \text{load processed rows} \longrightarrow \text{fit } \boldsymbol{\alpha} \longrightarrow \text{save model}\,,\\ \text{load calibration rows} \longrightarrow \text{calculate calibration scores} \longrightarrow \text{choose threshold}\,,\\ \text{load later rows} \longrightarrow \text{score each window} \longrightarrow \text{record decisions and evaluate labels.}\end{array}}$$

Packet preprocessing is intentionally outside the runtime analysis. None of the results below includes reading PCAP files, classifying packets, or building ten-minute windows. More importantly, no single reported number times every remaining box in this diagram. The benchmark separates model fitting, one training-matrix likelihood call, and one single-window score so that the source of a speed difference can be understood.

III.1 The four timing scopes that must not be confused

Table 6 states where each timer starts and stops. The first three scopes belong to the production benchmark. The last is the timer used by the professor's original fitting code.

The benchmark command does run isolated training, calibration, and scoring paths for both backends so that their models, thresholds, scores, and decisions can be compared. That execution is a correctness check. Its combined wall time is not reported as a pipeline benchmark. Therefore the phrase "complete model fit" should be used instead of "complete pipeline" when presenting the numbers below.

Table 6: Meaning of each runtime measurement.

Measurement	Included	Excluded
Complete model fit	Initial alpha construction; the initial matrix likelihood; every row-wise Minka digamma update; one matrix likelihood after each update; convergence checks; construction of the fit result.	PCAP preprocessing; loading processed JSON; construction of the backend and loading its coefficient tables; saving the model; calibration; later scoring; evaluation.
Training-matrix likelihood	One call to `backend.calculate(training.counts, reference_alpha)`. It includes Python validation, conversion from alpha to probabilities and ψ, the backend calculation, and return to Python.	Alpha initialization; Minka updates; repeated convergence iterations; the count-only multinomial coefficient; file input and output.
Single-window score	The complete log-probability for one representative non-silent window: its multinomial coefficient plus one one-row backend likelihood, followed by the configured raw-score return.	Loading the window; threshold comparison; event construction; writing output; calibration and label evaluation.
Original capsule's printed time	The accumulated duration of two likelihood calls in every Minka iteration: one at the old alpha and one at the candidate alpha.	The Minka digamma update, alpha and probability preparation, dataset loading, initialization outside those calls, and all other wall time.

III.2 Initial likelihood, iterative likelihood, and scoring likelihood

The word "likelihood" appears at three different moments in the project, so it is useful to follow one fit from the beginning.

First, the project creates an initial alpha vector from the category totals and the configured initial concentration. The likelihood does not calculate this alpha vector. After alpha has been initialized, the estimator calculates one initial likelihood and stores it as the baseline for convergence.

Next, every Minka iteration calculates a new alpha vector from digamma differences over the individual rows. LM and SciPy do not perform this update; both versions use the same SciPy digamma calculation. The selected backend then evaluates the row-wise likelihood at the new alpha. The estimator compares that value with the likelihood retained from the preceding step. If their absolute difference is below the tolerance, fitting stops. Otherwise the new value becomes the previous value for the next iteration.

In simplified form, the production estimator does this:

```
alpha = initialize_alpha(training_rows)
previous = matrix_likelihood(training_rows, alpha)  # one initial call

for each Minka iteration:
    alpha = digamma_fixed_point_update(training_rows, alpha)
    current = matrix_likelihood(training_rows, alpha)  # one call
    if abs(current - previous) < tolerance:
        stop
    previous = current
```

Thus a fit lasting I iterations performs $I + 1$ matrix-likelihood calls: one before the loop and one after each update. The likelihood guides the stopping decision; the digamma formula produces alpha.

Much later, during anomaly detection, the fitted alpha is fixed. A new window is assigned the full log-probability

$$\log P(\mathbf{x} \mid \boldsymbol{\alpha}) = \log \frac{N!}{\prod_k x_k!} + K(\mathbf{x}; \boldsymbol{\alpha}).$$

Here K is the backend likelihood kernel explained in Section II. The count-only multinomial coefficient is added for scoring because different windows are being compared. This scoring calculation does not update alpha and does not run Minka iterations.

The stored initial and final likelihoods are numerical values, not timing measurements. Table 7 shows the current LM fits and also explains why absolute fit times cannot be compared across devices without considering both matrix size and iteration count.

These likelihood values are sums of the parameter-dependent kernel across the training rows. Their magnitudes depend on the data and row counts, so they should not be compared as if a less negative value identified the better device model. Within one fit, the change is used only to decide when the alpha sequence has settled.

Table 7: Production training workloads and stored LM convergence values.

Dataset	Matrix	Iterations	Initial likelihood	Final likelihood
D-Link camera	846×4	1,882	−477,950.796	−476,052.589
Chromecast	867×4	1,950	−1,464,261.145	−1,462,542.173
Samsung Camera	432×4	580	−191,965.774	−191,562.039

III.3 How the production benchmark was measured

The production comparison gives LM and SciPy the same processed count matrix, the same estimator settings, and the same scoring workload. Backend objects are constructed before the timers start, so LM's one-time shared-library load and coefficient-table initialization are not charged to a likelihood call or fit. This is appropriate for a service that initializes its backend once and then uses it repeatedly, but it should be stated explicitly.

Complete fitting is measured five times per backend and reported as the median. The isolated likelihood and score operations are warmed once, then measured in five batches. Each likelihood batch contains 200 calls and each score batch contains 1,000 calls; the reported value is the median per-call duration. The standalone likelihood and score comparisons use the alpha fitted by the SciPy training run for both backends. This prevents a tiny backend-dependent alpha difference from changing the timed workload.

The cross-dataset measurements in the full project report were collected on 9 September 2026 on the same x86-64 development environment, using Python 3.12.14, NumPy 2.5.1, and SciPy 1.18.0. They are development-machine results, not measurements from an edge device, and normal system timing noise still applies.

III.4 Current production results: LM compared with SciPy

Figure 3 and Table 8 show the same three operations. Lower is better. Within each dataset, the LM/SciPy ratio is the useful comparison: values above one mean LM took longer and values below one mean LM took less time.

The current LM backend is slower than SciPy for the complete row-wise training likelihood on every dataset in this run. It is correspondingly slower for the complete fit. The size of the full-fit difference does not exactly equal the matrix-likelihood difference because the fit also spends time in alpha initialization and in the shared SciPy digamma updates. Samsung is fastest in absolute terms because it has only 432 training rows and reaches convergence in about 580 iterations; this does not mean its individual likelihood operation is the fastest relative to SciPy.

The result reverses for single-window scoring. LM takes about 12.0–12.2 microseconds and SciPy about 13.2 microseconds, making LM roughly 8–9 percent faster on this workload. A score calls the backend on only one row, where LM's native evaluation is competitive and there is no large matrix traversal. This is the operation closest to the detector's repeated online work, although the measurement still excludes packet capture, window construction, threshold comparison, and output.

An independently stored D-Link run produced 0.19297 seconds versus 0.18129 seconds for the complete fit, 55.46 versus 49.96 microseconds for the matrix likelihood, and 12.04 versus 13.39

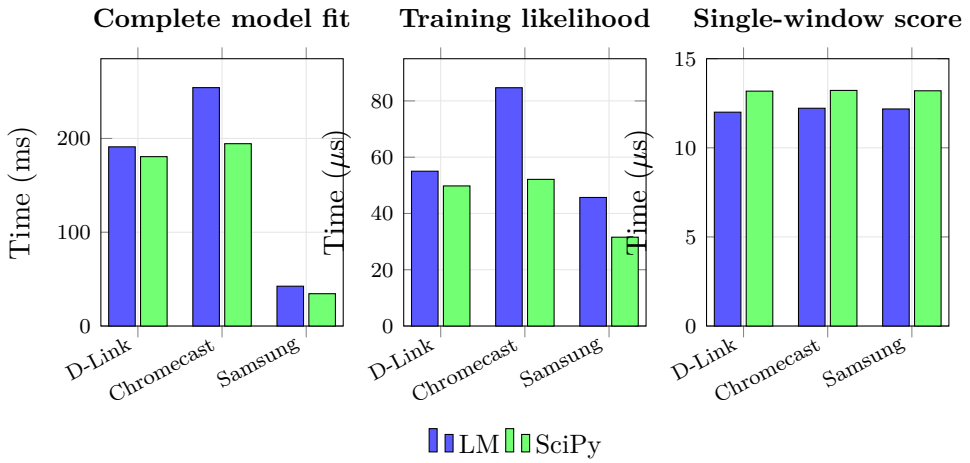


Figure 3: Median runtime for the three separately timed production operations. The fit uses an already loaded matrix, the training likelihood is one matrix call at fixed alpha, and the score is one complete single-window score.

Table 8: Production LM/SciPy runtime comparison. A positive difference means LM was slower; a negative difference means LM was faster.

Operation	Dataset	SciPy	LM	LM/SciPy	Difference
Complete fit	D-Link	180.59 ms	191.04 ms	1.058	+5.8%
	Chromecast	194.40 ms	254.17 ms	1.307	+30.7%
	Samsung	34.47 ms	42.46 ms	1.232	+23.2%
Matrix likelihood	D-Link	49.79 μs	55.02 μs	1.105	+10.5%
	Chromecast	52.12 μs	84.69 μs	1.625	+62.5%
	Samsung	31.57 μs	45.69 μs	1.447	+44.7%
Single score	D-Link	13.18 μs	12.00 μs	0.910	−9.0%
	Chromecast	13.22 μs	12.22 μs	0.924	−7.6%
	Samsung	13.20 μs	12.18 μs	0.923	−7.7%

microseconds for a score. These values differ slightly from the cross-dataset run because they were recorded in a separate timing session, but they support the same conclusion: current LM is close to SciPy for this complete fit, slower for the large matrix call, and faster for one-window scoring. The independent D-Link fit is 6.4 percent slower, which is inside the 10 percent "no significant speed difference" criterion used in the professor's IoT study. Chromecast and Samsung exceed that margin in the cross-dataset production run, so the criterion should not be generalized to all three fits.

III.5 What the optimization changed

Three code states appear in the discussion, and their names should be kept precise. The reference files under `lm-algorithm/code/` remain untouched; all application changes were made in the project's runtime copy.

Table 9: The three LM implementations referred to in the runtime discussion.

Code state	Likelihood semantics	Execution pattern
Original professor capsule	Sum the count rows first and evaluate one pooled vector for convergence.	Scalar LM entry point; old and candidate likelihood both recalculated each iteration; printed time contains only those calls.
Earlier project adapter	Evaluate every row independently and then sum the likelihood values.	Matrix wrapper calls the professor's complete scalar setup and evaluation sequence for every row.
Current project adapter	The same row-wise likelihood as the earlier project adapter.	Matrix wrapper performs common setup once, then retains separate validation and likelihood evaluation for every row.

The most important before-and-after comparison is between two versions of the project's row-wise LM adapter. It is not a comparison between the untouched professor capsule and the production detector. Both versions below calculate the same row-wise matrix likelihood. The earlier adapter safely called the professor's single-vector setup and evaluation sequence once per row. The optimized adapter prepares the alpha- and precision-dependent state once, retains all row-specific checks and evaluations, and releases the workspace at the matrix boundary.

Table 10: Controlled D-Link before-and-after result for the row-wise project implementation.

Operation	SciPy	Earlier project LM	Current project LM	LM reduction
846×4 likelihood	49.96 μs	114.68 μs	55.58 μs	51.5%
Complete model fit	0.18129 s	0.30410 s	0.19486 s	35.9%

Before this optimization, the LM matrix call was about 2.30 times the SciPy time and the complete fit was about 1.68 times the SciPy time. Afterwards the ratios were about 1.11 and 1.07. The likelihood speedup is larger than the fit speedup because only the LM likelihood portion was changed; Minka's digamma work remained the same.

The old and optimized project libraries produced bit-for-bit identical results over 1,000 deterministic stress matrices. Therefore the reduction measures the removal of repeated setup, not a relaxation

of precision, a change in the row-wise objective, or fewer Minka iterations. This is the strongest runtime improvement attributable directly to the native-code modification.

III.6 Why the original professor timing gives a different answer

The original capsule uses a different calling pattern. Its Minka update still reads every row, but it first sums the rows into one count vector for the convergence likelihood. During each iteration it calculates the likelihood at both the old and candidate alpha values:

```
for each Minka iteration:
    candidate_alpha = digamma_fixed_point_update(all_rows, old_alpha)
    start timer
    old_value = likelihood(sum(all_rows), old_alpha)
    stop timer
    start timer
    new_value = likelihood(sum(all_rows), candidate_alpha)
    stop timer
    add both durations to reported likelihood time
```

The old likelihood in one iteration is normally the new likelihood from the preceding iteration, but the capsule calculates it again. Its printed `time` is the sum of these two timed calls, not the elapsed time from the start to the end of fitting. By comparison, the production estimator retains the preceding value and performs one new row-wise likelihood call per iteration. The original capsule also uses a moment-based initial alpha and changes any row containing a zero by adding one to all its categories. The production estimator starts from observed category totals at a configured concentration and accepts zero counts directly. Finally, the capsule expresses much of the update as Python loops, whereas the production estimator uses NumPy array operations. The two programs therefore differ in row semantics, number of likelihood calls, initialization, zero handling, implementation of the update, and timer scope. Their raw printed times cannot be placed in one table as though they measured the same operation.

To make the professor-style architecture measurable, the project reproduced that calling pattern on deterministic samples from seven D-Link days. The experiment records both the accumulated likelihood-call time, matching the meaning of the capsule's timer, and a separate wall timer around the iteration loop. That wall timer starts after the rows have been aggregated and the moment-based initial alpha has been constructed. It includes the digamma updates, both likelihood calls, convergence checks, and loop overhead, but not data loading or those two setup steps. Each percentage below compares LM and SciPy within that same aggregate architecture. The reproduction uses the project's vectorized update and backend interfaces, so it reproduces the professor-style statistical and calling structure rather than timing the untouched capsule line for line.

In that experiment, LM is faster: it reduces the time spent in aggregate likelihood calls by 12.2 percent and the iteration-loop wall time by 8.8 percent. The same five samples fail to converge with both backends, so the difference is not created by one backend abandoning extra fits. On the complete 846×4 D-Link matrix, one pooled-vector call takes 8.44 microseconds with LM and 9.72 microseconds with SciPy, and a literal professor-style fit loop takes 0.15909 seconds with LM versus 0.16425 seconds with SciPy.

Table 11: Professor-style aggregate experiment. Each time is the mean total for fitting seven daily samples across three deterministic repeats.

Sample	Converged	Likelihood-call time			Iteration-loop wall time		
		SciPy	LM	Reduction	SciPy	LM	Reduction
10%	17/21	3.5746 s	3.1396 s	12.2%	4.8933 s	4.4593 s	8.9%
15%	20/21	1.9094 s	1.6766 s	12.2%	2.6517 s	2.4187 s	8.8%
20%	21/21	1.6485 s	1.4447 s	12.4%	2.3125 s	2.1064 s	8.9%
25%	21/21	0.9779 s	0.8599 s	12.1%	1.3957 s	1.2799 s	8.3%
All 84 fits	79/84	24.3311 s	21.3625 s	12.2%	33.7594 s	30.7930 s	8.8%

Those results do reproduce the runtime advantage of LM under the professor's aggregate calling pattern. They do not contradict the production results. Pooling changes an 846-row likelihood into one vector evaluation and therefore removes the workload on which SciPy's vectorization is strongest. It also changes the convergence objective, as explained in Section II. The aggregate experiment is evidence about the reference architecture; the row-wise benchmark is evidence about the anomaly detector actually implemented here.

III.7 Runtime must be read together with numerical agreement

A faster result is useful only if the two backends still implement the same calculation closely enough for the application. In the stored D-Link production benchmark, the maximum relative difference between fitted alpha values was 3.86×10^{-7}, the maximum absolute score difference was 1.09×10^{-5}, and the threshold difference was 2.89×10^{-6}. Both backends flagged the same three windows, with zero decision mismatches across 270 development windows. D-Link is unlabeled, so this demonstrates backend agreement rather than detection accuracy.

The same practical check was made when the three versioned device configurations were regenerated with LM after previously using SciPy. Across their 1,300 development windows, the largest relative alpha difference was 1.71×10^{-6}, the largest absolute score difference was 1.78×10^{-4}, and every anomaly decision was retained: 3 D-Link, 17 Chromecast, and 52 Samsung Camera windows were flagged by both sets of artifacts. For Chromecast and Samsung those counts can be evaluated against labels elsewhere in the project; here they establish only that changing the numerical backend did not change a decision.

The current D-Link fit is not numerically extreme. At its concentration of about 122.22, or $\psi \approx 8.18 \times 10^{-3}$, comparison with 100-decimal-digit arithmetic gave an absolute error of 3.07×10^{-10} for SciPy and 3.91×10^{-10} for LM. Both are already highly accurate. The reason to retain LM becomes clearer much closer to the Multinomial boundary:

This table explains the actual tradeoff. On the current datasets, SciPy is already accurate and often faster for fitting a complete matrix. LM provides much better protection against cancellation at extremely small ψ, and the optimized native path is slightly faster for the repeated one-window score operation measured here.

Table 12: Absolute error of one aggregate likelihood against high-precision arithmetic as concentration increases and ψ decreases.

Concentration	ψ	SciPy error	LM error
122.22	8.18×10^{-3}	3.07×10^{-10}	3.91×10^{-10}
10^{8}	10^{-8}	2.50×10^{-7}	1.50×10^{-8}
10^{10}	10^{-10}	2.83×10^{-5}	5.65×10^{-8}
10^{12}	10^{-12}	2.02×10^{-3}	5.58×10^{-5}
10^{14}	10^{-14}	3.37×10^{-1}	1.03×10^{-3}

III.8 The runtime conclusion to present

The clearest account for the meeting is the following. Preprocessing was deliberately excluded. The project did not report one end-to-end time for loading, fitting, calibration, scoring, and evaluation. It separately measured the estimator fit on an already loaded training matrix, one row-wise training-matrix likelihood call at fixed fitted alpha, and one full anomaly score for a representative window.

The earlier row-wise LM adapter was slow because it repeated invariant native setup for every row. Reusing that setup cut its D-Link matrix-likelihood time by 51.5 percent and its complete-fit time by 35.9 percent without changing any result. After that optimization, LM is close to SciPy on the D-Link complete fit, slower on the three large matrix likelihoods and complete fits in the cross-dataset benchmark, and about 8–9 percent faster for one-window scoring.

The professor-style aggregate experiment remains a separate and valid result. Under that architecture, LM reduces accumulated likelihood time by 12.2 percent and iteration-loop wall time by 8.8 percent. It cannot be used as the runtime of the production row-wise detector because it pools the rows, performs two likelihood calls per iteration, and times a different calculation. The defensible claim is therefore conditional: the native optimization materially improved the project integration, and LM's speed relative to SciPy depends on the operation and calling architecture being measured.

Source trail

The fitted-ψ comparison uses the processed windows under `data/processed/`, the daily fits in each dataset's `reports/.../capture_statistics.json`, the stored training models under `artifacts/`, and the UNSW window annotations under `data/labels/`. Aggregate, benign-only, and attack-only values were fitted with the same row-wise LM estimator and settings used by the project; labels were joined to processed rows by window start time.

The production timing definitions follow `src/lm_idnet/evaluation/benchmark_stage.py`, with the D-Link audit record in `reports/D-LinkDayCam5/lm_backend_benchmark.json`. The three-dataset measurements are recorded in `docs/lm_idnet_full_project_report.tex`. The controlled native before-and-after figures come from `docs/lm_backend_change_record.tex`. The aggregate architecture, including its separate likelihood and wall timers, is implemented in `scripts/lm_professor_experiment.` and recorded in `reports/D-LinkDayCam5/lm_professor_experiment.json` and `docs/lm_professor_architectu`

The descriptions of the adaptations follow `docs/lm_backend_change_record.tex`. The original capsule functions described above are in `lm-algorithm/code/LogL-global-v5.py` and its native files, especially `LM_time_lib.c`. The project path is visible in `src/lm_idnet/models/modeling_stage.py`, `src/lm_idnet/algorithms/dirichlet.py`, `src/lm_idnet/algorithms/log_likelihood.py`, `src/lm_idnet/alg` and the native LM source under `src/lm_idnet/algorithms/native/`.